GDB Tricks for CTF Pwn - 堆溢出利用

目录

- 基础命令
- 断点设置
- 查看内存和变量
- 修改内存和变量
- 堆相关命令
- 栈相关命令
- PIE保护绕过
- GDB脚本
- 常用插件

基础命令

```
# 启动调试
gdb ./binary
gdb -q ./binary              # 安静模式,不显示版权信息
gdb --args ./binary arg1 arg2 # 带参数启动

# 运行程序
r                            # run,运行程序
r < input.txt                # 从文件输入
r <<< $(python -c 'print "A"*100')  # 从命令输出输入

# 继续/单步执行
c                            # continue,继续执行
n                            # next,单步执行(不进入函数)
ni                           # next instruction,单步执行一条汇编指令
s                            # step,单步执行(进入函数)
si                           # step instruction,单步执行一条汇编指令(进入call)
finish                       # 运行到当前函数返回

# 退出
q                            # quit,退出gdb
```

断点设置

常规断点

```
# 函数断点
b main                          # 在main函数处下断点
b *main                         # 在main函数入口处下断点
b malloc                        # 在malloc函数处下断点
b free                          # 在free函数处下断点

# 地址断点
b *0x400566                     # 在指定地址下断点
b *$rebase(0x1234)              # PIE开启时,相对基址下断点(需要pwndbg)

# 条件断点
b malloc if $rdi==0x21          # 当malloc参数为0x21时断点
b *0x400566 if $rax==0          # 当rax为0时断点

# 临时断点
tb main                         # 临时断点,触发一次后自动删除
tbreak *0x400566               # 临时断点

# 断点管理
info b                          # 查看所有断点
delete 1                        # 删除断点1
delete                          # 删除所有断点
disable 1                       # 禁用断点1
enable 1                        # 启用断点1
```

PIE保护下的断点

```
# 方法1: 先运行再下断点
start                          # 启动程序并停在入口
b *$rebase(0x1234)            # 使用pwndbg的rebase功能

# 方法2: 计算实际地址
info proc mappings             # 查看内存映射,找到程序基址
b *0x555555554000+0x1234      # 基址+偏移

# 方法3: 使用符号
b main                         # 直接使用符号名
```

查看内存和变量

寄存器

```
info registers                 # 查看所有寄存器
i r                            # 简写
info registers rax rbx         # 查看特定寄存器
p $rax                         # 打印rax的值
p/x $rax                       # 以16进制打印
p $rip                         # 查看指令指针
```

内存查看

```
# examine命令格式: x/[数量][格式][大小] 地址
x/10gx 0x602000              # 查看10个8字节,16进制格式
x/20wx $rsp                  # 查看20个4字节,16进制格式
x/s 0x400000                 # 以字符串格式查看
x/i $rip                     # 以指令格式查看当前指令
x/10i $rip                   # 查看接下来10条指令

# 格式说明:
# x - 16进制  d - 十进制  u - 无符号十进制
# o - 八进制  t - 二进制  a - 地址
# c - 字符    s - 字符串  i - 指令

# 大小说明:
# b - byte(1字节)  h - halfword(2字节)
# w - word(4字节)  g - giant(8字节)

# 查看特定变量
p variable_name               # 打印变量
p/x variable_name             # 16进制打印
p &variable_name              # 打印变量地址
p *ptr                        # 解引用指针
p array[0]@10                 # 打印数组的10个元素
```

反汇编

```
disass main                   # 反汇编main函数
disass 0x400000              # 反汇编指定地址
disass $rip,$rip+20          # 反汇编指定范围
set disassembly-flavor intel # 设置Intel语法
```

修改内存和变量

```
# 修改寄存器
set $rax = 0                    # 设置rax为0
set $rip = 0x400566             # 修改指令指针

# 修改内存
set {long}0x602000 = 0x41414141    # 修改8字节
set {int}0x602000 = 0x41414141     # 修改4字节
set {short}0x602000 = 0x4141       # 修改2字节
set {char}0x602000 = 0x41          # 修改1字节

# 修改变量
set variable var_name = 100        # 修改变量值
set variable ptr = 0x602000        # 修改指针

# 批量写入
set {long[4]}0x602000 = {0x41, 0x42, 0x43, 0x44}
```

堆相关命令

基础堆命令 (需要pwndbg/gef/peda)

pwndbg命令

```
# 查看堆状态
heap                          # 查看所有堆块
arena                         # 查看当前arena信息
arenas                        # 查看所有arena
bins                          # 查看所有bins状态
fastbins                      # 查看fastbins
unsortedbin                   # 查看unsorted bin
smallbins                     # 查看small bins
largebins                     # 查看large bins
tcache                        # 查看tcache (glibc >= 2.26)

# 查看堆块详细信息
heap -v                       # 详细模式查看堆
vis_heap_chunks               # 可视化显示堆块
vis_heap_chunks 0x602000      # 从指定地址开始显示
dq 0x602000 100               # 查看堆内存(dump qwords)

# 查看特定堆块
malloc_chunk 0x602000         # 查看指定地址的chunk结构
print *(struct malloc_chunk*)0x602000  # 打印chunk结构体
```

查看堆结构偏移

```
# 查看chunk结构
p *(struct malloc_chunk*)0x602000
# 输出:
# {
#   prev_size = 0x0,
#   size = 0x21,
#   fd = 0x0,
#   bk = 0x0
# }

# 查看tcache结构 (glibc >= 2.26)
p *(tcache_perthread_struct*)tcache
p *(tcache_entry*)0x602000

# 手动计算偏移
x/4gx 0x602000-0x10          # chunk头部在用户数据-0x10
x/2gx 0x602000               # fd, bk指针位置
```

查看main_arena

```
# 方法1: 直接查看符号
p main_arena                 # 打印main_arena结构
p &main_arena                # 查看main_arena地址
x/128gx &main_arena          # 查看main_arena内存

# 方法2: 通过libc基址计算
vmmap                        # 查看内存映射,找到libc基址
p *(struct malloc_state*)(libc_base + main_arena_offset)

# 查看arena结构成员
p main_arena.top             # top chunk地址
p main_arena.last_remainder  # last remainder地址
p main_arena.bins            # bins数组
p main_arena.bins[0]@254     # 打印所有bins

# arena偏移 (glibc版本相关)
# glibc-2.23: offset = 0x3c4b20
# glibc-2.27: offset = 0x3ebc40
# glibc-2.31: offset = 0x1ecb80
```

查看bins详细信息

```
# fastbins (0x20-0x80)
fastbins                      # pwndbg命令
p main_arena.fastbinsY        # 直接查看fastbins数组
p main_arena.fastbinsY[0]@10 # 查看所有fastbins

# unsorted bin
unsortedbin                   # pwndbg命令
p main_arena.bins[0]          # unsorted bin的fd
p main_arena.bins[1]          # unsorted bin的bk

# small bins (< 0x400)
smallbins                     # pwndbg命令
p main_arena.bins[2]@124      # small bins在bins[2]开始

# large bins (>= 0x400)
largebins                     # pwndbg命令
p main_arena.bins[126]@126    # large bins在bins后面

# tcache (glibc >= 2.26)
tcache                        # pwndbg命令
tcachebins                    # 查看所有tcache bins
p *(tcache_perthread_struct*)tcache
```

堆块遍历

```
# 从heap基址遍历
set $chunk = 0x602000
while $chunk != 0
  x/4gx $chunk
  set $chunk = $chunk + (*(long*)($chunk+8) & ~0xf)
end
```

栈相关命令

查看栈信息

```
# 查看栈指针
p $rsp                          # 栈顶指针
p $rbp                          # 栈基址指针

# 查看栈内存
x/40gx $rsp                     # 查看栈顶40个8字节
x/40gx $rbp                     # 从rbp开始查看
stack 40                        # pwndbg命令,查看40行栈
telescope $rsp 40               # pwndbg命令,智能解析栈数据

# 查看调用栈
bt                              # backtrace,查看函数调用栈
bt full                         # 显示所有局部变量
frame 0                         # 切换到栈帧0
info frame                      # 查看当前栈帧信息
info locals                     # 查看局部变量
info args                       # 查看函数参数
```

查看栈结构偏移

```
# 查看局部变量偏移
info frame                      # 查看栈帧信息
p &local_var                    # 查看局部变量地址
p $rbp - &local_var             # 计算相对rbp的偏移

# 查看返回地址
x/gx $rbp+8                     # 返回地址位置 (x64)
x/wx $ebp+4                     # 返回地址位置 (x86)

# 查看保存的rbp
x/gx $rbp                       # saved rbp位置

# canary位置 (如果有)
x/gx $rbp-8                     # 通常在saved rbp前面
```

栈溢出检查

```
# 查找canary
search -8 <canary_value>      # 在内存中搜索canary
telescope $rsp                # 查看栈,canary会高亮显示

# 计算溢出偏移
cyclic 200                    # 生成200字节的循环模式
cyclic -l 0x61616161          # 查找模式偏移
```

PIE保护绕过

查找程序基址

```
# 方法1: info proc mappings
info proc mappings            # 查看内存映射
# 第一个可执行段的地址就是基址

# 方法2: vmmap (pwndbg)
vmmap                         # 彩色显示内存映射
vmmap binary_name             # 只显示程序相关映射

# 方法3: 通过寄存器计算
p $rip                        # 查看当前指令地址
p $rip & ~0xfff               # 页对齐后的基址(粗略)

# 方法4: 查看/proc/self/maps
shell cat /proc/$pid/maps     # 查看进程内存映射
```

PIE下计算实际地址

```
# 假设程序基址为0x555555554000,IDA中地址为0x1234
set $base = 0x555555554000
p/x $base + 0x1234            # 计算实际地址
b *($base + 0x1234)           # 在实际地址下断点

# 使用pwndbg的rebase (推荐)
piebase                       # 查看PIE基址
b *$rebase(0x1234)           # 自动计算实际地址
x/10i $rebase(0x1234)        # 查看指令
```

查找函数地址

```
# 即使开启PIE,符号依然可用
b main                        # 直接使用符号
p main                        # 查看main函数地址
p &main                       # 同上
disass main                   # 反汇编main

# 查看GOT/PLT
got                           # pwndbg命令,查看GOT表
plt                           # 查看PLT表
x/gx &'free@got.plt'          # 查看GOT表项
p 'free@plt'                  # 查看PLT地址
```

GDB脚本

命令脚本 (.gdb文件)

```
# script.gdb
set pagination off
set breakpoint pending on

# 定义断点
b malloc
commands
  silent
  printf "malloc(%p)\n", $rdi
  c
end

b free
commands
  silent
  printf "free(%p)\n", $rdi
  x/4gx $rdi-0x10
  c
end

# 运行
r
```

使用脚本

```
gdb -x script.gdb ./binary   # 启动时加载脚本
source script.gdb            # gdb中加载脚本
```

Python脚本 (gdb.python)

```python
# exploit.py
import gdb

class MallocBreakpoint(gdb.Breakpoint):
    def __init__(self):
        super(MallocBreakpoint, self).__init__("malloc")

    def stop(self):
        size = gdb.parse_and_eval("$rdi")
        print(f"malloc(0x{int(size):x})")
        return False  # 不停止,继续运行

# 创建断点
MallocBreakpoint()

# 运行程序
gdb.execute("run")
```

在GDB中使用Python

```
# 内联Python
python print("Hello from Python")
python gdb.execute("info registers")

# 加载Python脚本
source exploit.py

# Python交互模式
python-interactive
>>> import gdb
>>> gdb.execute("x/10gx $rsp")
```

自动化脚本示例

```
# auto_exploit.gdb
# 自动化堆溢出利用脚本

set pagination off
set follow-fork-mode parent
file ./binary

# 第一次malloc后断点
b malloc
commands 1
  silent
  set $chunk1 = $rax
  printf "chunk1: %p\n", $chunk1
  c
end

# 第二次malloc后断点
b malloc
commands 2
  silent
  set $chunk2 = $rax
  printf "chunk2: %p\n", $chunk2
  # 删除这个断点
  delete 2
  c
end

# free前断点
b free
commands
  silent
  printf "freeing: %p\n", $rdi
  x/4gx $rdi-0x10
  c
end

r < input.txt
```

条件日志脚本

```
# log_chunks.gdb
set logging file chunks.log
set logging on
set pagination off

b malloc
commands
  silent
  finish
  printf "malloc(0x%lx) = %p\n", $rdi, $rax
  c
end

b free
commands
  silent
  printf "free(%p)\n", $rdi
  x/4gx $rdi-0x10
  c
end

r
set logging off
```

常用插件

pwndbg (推荐)

```
# 安装
git clone https://github.com/pwndbg/pwndbg
cd pwndbg
./setup.sh

# 常用命令
context                         # 显示上下文(寄存器/代码/栈/回溯)
heap                            # 堆分析
bins                            # 查看所有bins
cyclic 200                      # 生成循环模式
cyclic -l 0x61616161            # 查找偏移
vmmap                           # 内存映射
search                          # 搜索内存
telescope                       # 智能解析内存
piebase                         # PIE基址
```

gef (GDB Enhanced Features)

```
# 安装
bash -c "$(curl -fsSL https://gef.blah.cat/sh)"

# 常用命令
gef config                      # 配置
heap chunks                     # 查看堆块
heap bins                       # 查看bins
pattern create 200              # 生成模式
pattern search 0x61616161       # 搜索模式
vmmap                           # 内存映射
search-pattern                  # 搜索模式
```

peda (Python Exploit Development Assistance)

```
# 安装
git clone https://github.com/longld/peda.git ~/peda
echo "source ~/peda/peda.py" >> ~/.gdbinit

# 常用命令
context                      # 显示上下文
pattern create 200           # 生成模式
pattern offset 0x61616161    # 查找偏移
checksec                     # 检查保护
vmmap                        # 内存映射
find                         # 搜索内存
```

插件配置文件 ~/.gdbinit

```
# .gdbinit 示例
source ~/pwndbg/gdbinit.py   # 加载pwndbg

# 通用设置
set disassembly-flavor intel
set pagination off
set print pretty on
set history save on
set history filename ~/.gdb_history

# 自定义命令
define hook-stop
  context
end
```

高级技巧

查找gadgets

```
# 使用ROPgadget (外部工具)
# 在gdb中查找特定指令
find /b 0x400000, 0x401000, 0x5f, 0xc3  # 查找 "pop rdi; ret"
x/2i 0x400566                            # 验证gadget

# 搜索字符串
search "/bin/sh"
search -t string "/bin/sh"
```

dump内存

```
# dump到文件
dump binary memory output.bin 0x602000 0x603000
dump binary value output.bin variable_name

# 从文件恢复
restore output.bin binary 0x602000
```

调试子进程

```
set follow-fork-mode child   # 跟踪子进程
set follow-fork-mode parent  # 跟踪父进程(默认)
set detach-on-fork off       # 不分离,可同时调试父子进程
info inferiors               # 查看所有进程
inferior 2                   # 切换到进程2
```

远程调试

```
# 在目标机器上
gdbserver :1234 ./binary

# 在本地
gdb ./binary
target remote ip:1234
```

实战案例脚本

UAF (Use After Free) 调试脚本

```
# uaf_debug.gdb
b malloc
commands
  finish
  printf "malloc() = %p\n", $rax
  c
end

b free
commands
  printf "free(%p)\n", $rdi
  x/4gx $rdi-0x10
  set $freed = $rdi
  c
end

# 当程序再次使用freed地址时断点
watch *$freed

r
```

Double Free检测脚本

```
# double_free.gdb
python
freed_chunks = set()

class FreeBreakpoint(gdb.Breakpoint):
    def __init__(self):
        super().__init__("free")

    def stop(self):
        addr = int(gdb.parse_and_eval("$rdi"))
        if addr in freed_chunks:
            print(f"[!] Double Free detected: 0x{addr:x}")
            return True
        freed_chunks.add(addr)
        print(f"[*] Free: 0x{addr:x}")
        return False

FreeBreakpoint()
end

r
```

堆溢出覆盖检测

```
# heap_overflow.gdb
# 在strcpy等函数后检查chunk完整性
b strcpy
commands
  finish
  printf "strcpy dest: %p\n", $rdi
  x/10gx $rdi-0x10
  # 检查下一个chunk的size字段是否被破坏
  set $next_size = *(long*)($rdi + *(long*)($rdi-8))
  if ($next_size & 0xfff) == 0
    printf "[!] Possible heap overflow detected!\n"
  end
  c
end

r
```

快捷键和别名

GDB快捷键

```
Ctrl+C                          # 中断程序
Ctrl+D                          # 退出gdb
Ctrl+L                          # 清屏
Ctrl+P                          # 上一条命令
Ctrl+N                          # 下一条命令
```

自定义别名 (在.gdbinit中)

```
# 简化命令
define bpl
  info breakpoints
end

define ctx
  context
end

define gg
  set $base = (long long)$arg0
  b *($base + $arg1)
end
# 使用: gg 0x555555554000 0x1234
```

调试技巧总结

1. **开始调试前**: 使用 `checksec` 检查保护, `file` 查看文件类型
2. **设置断点**: 在关键函数(malloc/free/strcpy)下断点
3. **查看堆状态**: 使用 `heap` 和 `bins` 命令观察堆布局
4. **PIE程序**: 使用 `vmmap` 查找基址,使用 `$rebase()` 下断点
5. **脚本自动化**: 编写gdb脚本自动化重复操作
6. **使用插件**: pwndbg/gef大幅提升调试效率
7. **日志记录**: 使用 `set logging on` 记录调试过程

参考资源

- pwndbg: https://github.com/pwndbg/pwndbg
- gef: https://github.com/hugsy/gef
- glibc source: https://sourceware.org/glibc/
- how2heap: https://github.com/shellphish/how2heap
- GDB Manual: https://sourceware.org/gdb/documentation/

最后更新: 2026-02-27
适用于: glibc 2.23-2.35, x86-64架构